VoiceOps 2026 — ZentroMart support agent

Team: Team VoiceOps · LLM & Generative AI Course, C&A Club IIT Guwahati × Gnani.ai

Repository: `https://github.com/sustik78/From-Tokens-to-Transformers-IITG`

1. What we built

One grounded support brain, exposed as a text answerer, a multi-turn agent and a voice pipeline. The same retrieval, policy handlers and tools are reused in all three; voice only adds Gnani STT at the front and Gnani TTS at the back.

```
customer audio -> Gnani STT -> SupportAgent -> grounded retrieval + tools
                                            |
                                            +-> Gnani TTS -> spoken reply
```

The knowledge base is split by `[SECTION-ID]`, retaining status, effective date and document type. Retrieval combines BM25 with multilingual sentence embeddings, then applies precedence (`current policy -> agent guide -> help -> FAQ -> marketing`) and intent-aware boosts. Superseded return policy v3 is indexed for history but cannot override v4. Policy numbers and dates come from the current sections; order facts come from the supplied CSV tables through the supplied tools.

The conversation layer remembers the order ID, verification state and the original issue. It asks one question at a time, answers the pending issue after verification, refuses out-of-policy requests and escalates when the documents require a human. `lookup_order` never reveals details before `verified=True`; a ticket is claimed only after `create_ticket` returns `created=True`.

2. Step 1 vs Step 2

Step 1 used a plain system prompt and the same 52 `asr_text` questions, with no documents and no tools. We used Pollinations `openai-fast` after the free LLM7 daily quota was exhausted. Step 2 used the RAG answerer with the current corpus, hybrid retrieval and the dataset date rules.

Metric (52 questions)	Step 1: prompt only	Step 2: RAG
Answer must-include pass	8 / 49 (16.3%)	49 / 49 (100%)
Joint answer + decision pass	8 / 52 (15.4%)	52 / 52 (100%)
Decision match	46 / 52 (88.5%)	52 / 52 (100%)
Gold evidence in retrieved chunks	n/a	47 / 47 (100%)

The prompt-only model guessed plausible but wrong company facts: 30 days for electronics instead of 7, a 3–5 day wallet refund instead of instant/2 hours, a student discount that does not exist, and a 30-day return for a torn jeans item instead of the damaged-item route. RAG fixed the facts and, importantly, made the answer traceable: every answer stores the section IDs used.

Retrieval quality was measured against `gold_evidence`, including product SKUs. The final index reached 47/47 on the questions with gold evidence. Intent boosts were added only for document routing (for example warranty/assured, return windows, opened beauty items and delivery attempts), never for answer text.

3. Step 3: conversations, tools and refusals

`SupportAgent` is a small explicit state machine rather than an unconstrained

planner. On the first turn it identifies itself as AI. For order-record questions it asks for the order ID; when an ID is present it calls `lookup_order` unverified, then asks for the last four phone digits or delivery pincode. Two failed attempts lock order details. The original issue is stored before verification so the agent can answer it immediately after a successful check. Standalone policy questions are answered without forcing an order lookup.

On the 23 practice conversations (96 turns):

Check	Result
Conversations passing every turn	23 / 23 (100%)
Turns passing decision + reply + tools	96 / 96 (100%)
Decision match	96 / 96 (100%)
Must-include / must-not-include	77 / 77 (100%)
Expected tool calls	54 / 54 (100%)
Gold evidence on retrieval-answer turns	27 / 27 (100%)

High-value damage is handled honestly: the agent explains DWM-3/ESC-1, creates a ticket, and never says it approved a replacement. Shipped orders are not cancelled; the agent explains refusal-at-door or return instead. Repeat contacts, fraud, safety, privacy and explicit human requests go to a human with a short summary. Verification values are held only in private tool arguments and are removed from public result dictionaries.

4. Step 4: Gnani voice

`src/voice.py` uses only the official Gnani SDK. STT is `GnaniSTTClient.transcribe(..., language_code=...)`; TTS is `GnaniTTSClient.synthesize(..., model="timbre-v2.5", voice="Kaveri", ...)`. The model is named explicitly because the SDK's older default is no longer served.

The required batch command is:

```
python run.py --audio_dir <folder> --out_dir <folder> [--data_dir <dataset>]
```

For each clip it saves the transcript, reply text, reply WAV, retrieved chunk IDs, tool calls and a manifest. Five Gnani-generated test clips (four English, one Hindi) ran end to end, 5/5 with speech output. The Hindi clip is transcribed as `en-IN` Hinglish; the reply is spoken as `hi-IN`. Hinglish TTS is mapped to `hi-IN` because the installed SDK rejects `hi-en`.

Two robustness fixes came from real calls. Cloudflare occasionally returns 403 for one specific upload; the client retries three times and the batch continues. Hindi STT returns number words such as "chhah hazaar ... pachahattar"; the order-ID parser now converts Indian number words and spaced digit groups, so "refund kahaan hai" with a spoken order number reaches the verification step instead of "I don't know".

5. What went wrong, and what we fixed

• **Prompt-only hallucination.** The baseline invented plausible policies. RAG plus section metadata and explicit "no information" handling fixed this.

• **ASR number forms.** `ele-zero zero one to`, `to five six`, Hindi thousands and spoken 4+4 order IDs were normalised before intent detection and lookup.

• **Router order.** "Mera refund kahan hai?" was being read as "where is my order".

Refund status/timeline now wins when refund language is present.

• **Date edge cases.** Return windows use delivery date; business-day refunds skip Sundays and the four supplied holidays; festive-sale returns use the placed date.

• **A dangerous substring.** The return-fee pattern `"49"` matched order IDs such as `6649  2475`. It was removed and replaced with fee-language patterns.

• **Lost follow-ups.** The pending issue was overwritten on the order-ID turn. It is now captured only once, then answered after verification.

• **False ticket claims.** Ticket wording is tied to the tool's `created` flag.

• **Provider limits.** LLM7 hit a daily quota; Step 1 was rerun in four batches on Pollinations with the required 30-second spacing. Step 2/3 are offline.

6. Limitations and next steps

The free LLM used for the Step 1 baseline is rate-limited and not production-grade; Step 2/3 deliberately do not depend on it. The batch runner creates one agent per audio file, because the organizer contract is a folder of independent clips; the Streamlit demo keeps conversation state across turns. Hindi templates should get a native-speaker review before a real launch. There is no live telephony integration; the Gnani layer is exercised through WAV clips and the web demo. Next we would add a small cross-encoder reranker, cache verified sessions, and record a human demo call.

7. Reproduce

```
pip install -r requirements.txt
python -m src.eval_step1 --provider pollinations --batch-size 13 --delay 32
python -m src.eval_step2
python -m src.eval_step3
python run.py --audio_dir test_audio --out_dir sample_outputs
streamlit run src/app.py
```

The dataset is not redistributed. Set `DATA_DIR` or pass `--data_dir`; configure `GNANI_API_KEY` in `.env` only. `.env.example` contains no secrets.